

A Voyage to the Kernel



Part 25

Segment: 4.5, Day 24

In the last column, we discussed the USB subsystem, and ways to get around with USB drivers. Now let us focus our attention on a more important (in my perspective) subsystem—networking. While we were discussing the kernel's structure, we did spend some time on networking, but since the focus was on an overall view, we skipped many points specific to the subsystem. As I promised, we are going to revisit the subsystem and explore it in detail.

First, let's see how networking is initialised by the kernel. The file *main.c* contains the function *do_basic_setup(void)*, which is defined as shown in the following code block.

```
static void __init do_basic_setup(void)
{
    init_workqueues();
    cpuset_init_smp();
    usermodehelper_init();
    init_ttpfs();
    driver_init();
    init_irq_proc();
    doctors();
    do_initcalls();
}
```

The function *do_initcalls()* calls some functions that invoke other networking functions, in turn. You can find all the important files pertaining to networking under the *net* directory of the kernel tree. The first file that we are going to explore is *socket.c*. It contains many functions that perform network initialisation. A typical example is *sock_init(void)*, which invokes the subsystem during the booting process:

```
static int __init sock_init(void)
{
```

```
    sk_init();
    skb_init();

    /*
     *   Initialise the protocols module.
     */

    init_inodecache();
    register_filesystem(&sock_fs_type);
    sock_mnt = kern_mount(&sock_fs_type);

    /* The real protocol initialisation is performed in later
    initcalls.
    */

#ifdef CONFIG_NETFILTER
    netfilter_init();
#endif

    return 0;
}

The sk_init() function performs the initialisation of
the sock SLAB cache, and skb_init() initialises the skbuff
SLAB cache. It is important to note that the socket files
have some specific and generic operations. These won't
be appearing in the struct, but are done using socketcall().
Given below is the struct, for your reference:

static const struct file_operations socket_file_ops = {
    .owner = THIS_MODULE,
    .llseek = no_llseek,
    .aio_read = sock_aio_read,
    .aio_write = sock_aio_write,
    .poll = sock_poll,
    .unlocked_ioctl = sock_ioctl,
#ifdef CONFIG_COMPAT
    .compat_ioctl = compat_sock_ioctl,
```